A Map of Models

Unit 2: What Is on the Menu, and How to Choose

Stat 220 | Statistics for Data Science

Where we left off

- Unit 1 compared two groups. One estimate, one standard error, one question.
- Almost nothing you will be handed at work looks like that. You get a table with forty columns and a vague question attached.
- This unit is the map: what is available, what each one lets you do, and how to pick.

Principle: the honest starting point

"Which model should I use?" has no answer until you say what you want out of it. Most bad modeling starts with someone skipping that sentence.

Every model is the same sentence

$$y = f(x_1, x_2, \ldots, x_p) + \text{error}$$

- y is the thing you want to know. The x's are what you already have.
- f is the part the model supplies. Choosing a model is choosing what shapes f is allowed to take.
- The error is everything f does not account for. Some models make a claim about it. Others say nothing at all.

Principle: that is the whole menu

Every family in this unit is a different answer to two questions. What is f allowed to look like, and do we say anything about the error?

What this unit gives you

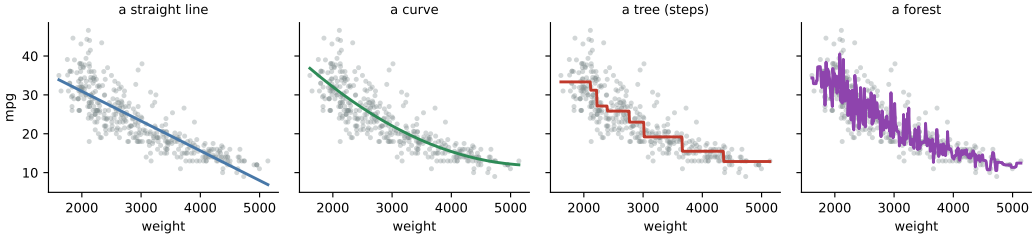
Things to know

- the main model families, and what each one assumes
- that the type of y decides most of the choice for you
- the five jobs people hand a model
- the five things you do to any model once you pick it
- how to measure error honestly, and compare two models

Mistakes to stop making

- picking the algorithm before naming the goal
- forcing the wrong model onto the type of y
- judging a model on the data you fit it to
- reading a penalized coefficient as an effect
- reaching for flexibility on a few hundred rows

What f can look like



- **A straight line.** One slope, and you can say it out loud in units.
- **A curve.** One extra term, still readable.
- **A tree.** A step function, which is a set of if-then rules.
- **A forest.** An average of hundreds of trees, chasing every bump here.

Same 392 cars, same predictor, four families. Only f changed.

Two kinds of thing get called a model

A probability model

Says where the data came from. It writes down a distribution:

$$y \sim \text{Normal}(b_0 + b_1 x,\ \sigma^2)$$

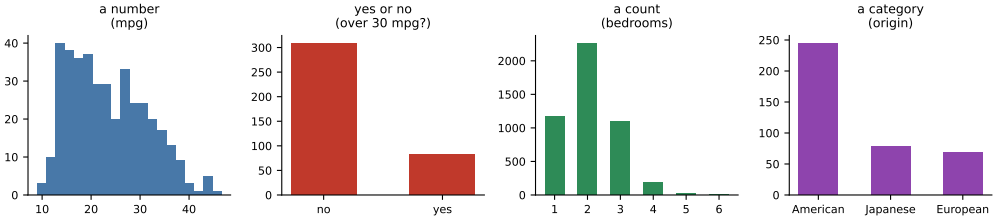
An algorithmic model

Says nothing about where the data came from. It gives you a rule that turns inputs into a prediction, tuned until the predictions are good.

Principle: why the split matters

A model that names a distribution has already said what *other* datasets like yours would have looked like, so it can tell you how much its own numbers would move. A model that only produces predictions cannot. To learn how much it would move, you have to hold data back and watch it miss.

Start with the type of y



This is the one part of the choice that is not a judgment call, and it removes most of the menu before you have thought about anything else.

What each type of y points to

y is	Example	Probability model	Trees, forests, boosting
a number	mpg, rent, revenue	linear regression	in regression mode
yes or no	churn, default, click	logistic regression	in classification mode
a count or rate	tickets per hour	Poisson, negative binomial	on counts
a category	which of four plans	multinomial logistic	in multi-class mode
time to an event	months until churn	survival models	specialized versions

Read the last column down. It is the **same three families every row**, and only the mode changes.

Trap: forcing the wrong type

Run a linear regression on a yes/no outcome and it produces numbers. Some will be predicted probabilities above 1 or below 0, which should tell you what happened.

Two filters, doing different jobs

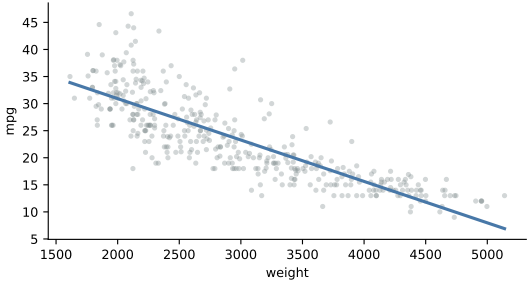
Notice what the right-hand column of that table just did. It repeated itself.

	Probability models	**Trees, forests, boosting**
Does the type of y narrow it?	yes, down to one	no, they all have a version for every type
So what rules it out?	the type of y	**the job**, never the outcome

Principle: the outcome never rules out a forest

Linear regression genuinely cannot take a count, and logistic genuinely cannot take a number. A forest will run on any of them. So when you decide against one, the reason is always that somebody needs to read a number out of the model, and a forest has none to give.

Linear regression

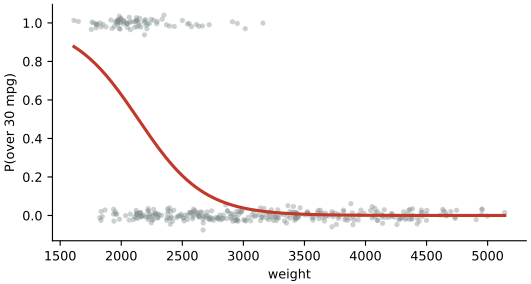


$$y \sim \text{Normal}(b_0 + b_1 x,\ \sigma^2)$$

- y is a **number**.
- f is a straight line, so one slope describes the whole relationship.
- You can read b_1 out loud in units and defend it.

Costs you: the world has to be roughly straight, or you have to bend it yourself.

Logistic regression

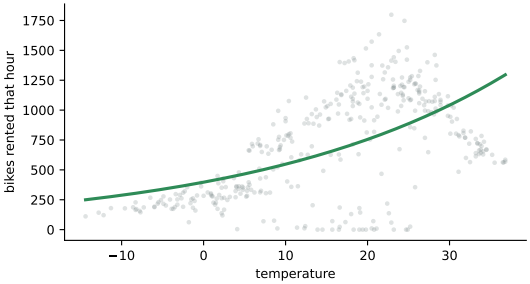


$$y \sim \text{Bernoulli}(p(x))$$

- y is **yes or no**. The output is a probability, never a label.
- f is a straight line on the log-odds, which becomes an S-curve on the probability.
- Predictions are trapped between 0 and 1 by construction.

Costs you: somebody still has to pick the cutoff for acting.

Poisson regression

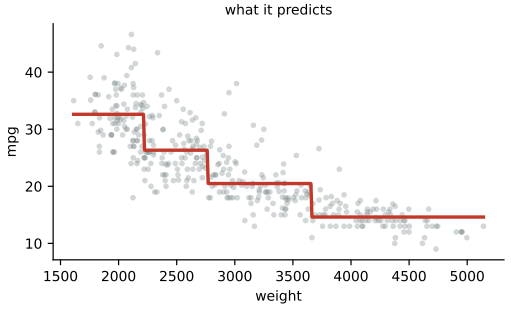


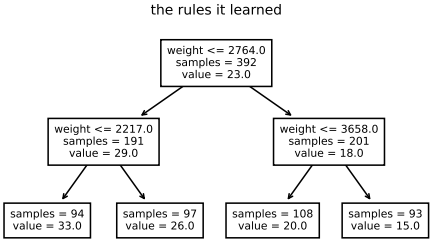
$$y \sim \text{Poisson}(\lambda(x))$$

- y is a **count**: bikes rented in an hour, tickets, complaints.
- f works on the log of the rate, so predictions can never go negative.
- Built in: counts are more variable when the rate is higher.

Costs you: it assumes the variance equals the mean, which real counts often exceed.

Decision tree



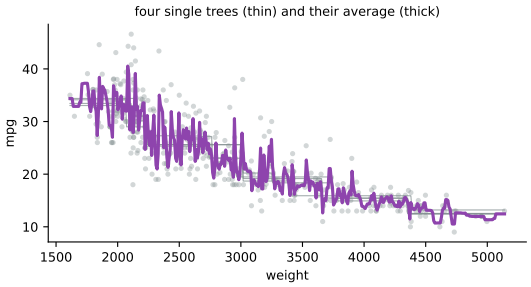


- Splits the data one variable at a time and predicts the average inside each box.
- f is a **step function**, and the same thing written out is a set of if-then rules a human can follow.
- No distribution is named anywhere, so there is nothing here to test.

Any y: the same algorithm splits on a number, a yes/no, or a count.

Costs you: move a few rows and the splits can land somewhere else entirely.

Random forest

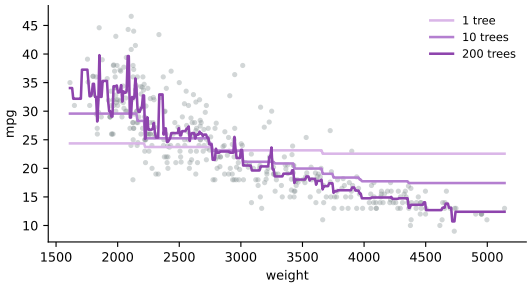


- Grow hundreds of trees on resampled data, then average them.
- Each single tree is unstable. Averaging cancels most of that out, which is the whole idea.
- Solid predictions with almost no tuning.

Any y: regression mode or classification mode, same idea.

Costs you: there is no line to read any more, and no coefficient to report.

Gradient boosting

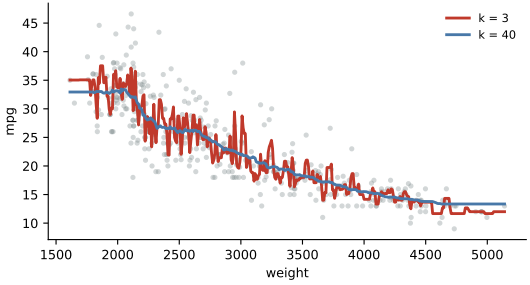


- Trees added one at a time, each one fitting what the previous ones got wrong.
- One tree is a crude step. Ten is closer. Two hundred traces the shape.
- Usually the most accurate thing you can fit to a table of numbers.

Any y: numbers, yes/no, counts, categories.

Costs you: several hyperparameters that matter, and it will overfit if you let it.

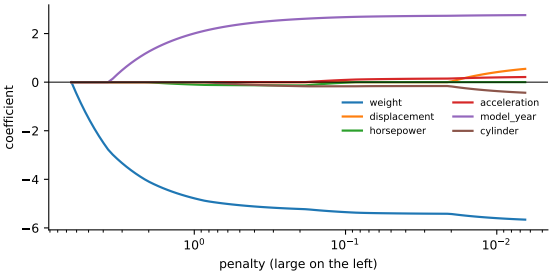
k-nearest neighbours



- To predict a new case, find the k most similar past cases and average them.
- There is no fitted equation at all. The training data *is* the model.
- k sets the smoothness: 3 is jumpy, 40 is smooth.

Costs you: it degrades badly once you have many columns, and it stores everything.

Lasso and ridge



- A linear regression with a penalty on large coefficients.
- Raise the penalty and coefficients slide toward zero. The lasso sends most of them to exactly zero, which is how it picks a shortlist.
- Useful when you have far more columns than you can defend.

Costs you: every coefficient is now deliberately biased, so they are not effects.

The menu, on one page

Model	**Names a distribution**	f **is**	**Reach for it when**
Linear regression	yes	a straight line	you must defend a number
Logistic regression	yes	an S-curve on a probability	y is yes or no and someone reads it
Poisson regression	yes	a curve on a rate	y is a count
Decision tree	no	a step function	you want followable rules
Random forest	no	an average of trees	prediction matters, reading does not
Gradient boosting	no	a sum of small trees	you want the best accuracy available
k-nearest neighbours	no	a local average	few columns, and similarity is meaningful
Lasso and ridge	partly	a penalized line	too many columns to defend

Five jobs people hand a model

1. **Predict.** Give me a number for this new case.
2. **Explain.** Does X actually move Y, and by how much?
3. **Screen.** Out of these 200 columns, which few are worth tracking?
4. **Decide.** Given what it costs to be wrong each way, what should we do?
5. **Deploy.** Run this every night for two years, and be able to explain any single output to someone who is unhappy about it.

Principle: say which one out loud first

A model that is excellent at the first can be useless at the second, and the choice that makes the fifth easy usually costs you accuracy on the first.

Job 1: predict

The ask. A listings site wants an estimated rent on every apartment page.

- **What matters:** error on apartments the model has never seen. Nothing else.
- **Reach for:** boosting or a forest when you have plenty of rows, a regression when rows are scarce.
- **Nobody will ever read a coefficient**, so nothing constrains you except accuracy and whatever you need to keep it running.

Trap: judging it on the wrong data

A model that predicts the rows it was fitted to is not predicting. It is remembering. The only number that counts here comes from data the model never saw.

Job 2: explain

The ask. A city council wants to know whether apartment size drives rent, and will write policy from your answer.

- **What matters:** one coefficient, its units, and an honest range around it.
- **Reach for:** linear or logistic regression. Only a probability model gives you a number you can defend.
- Accuracy is almost beside the point. A model that predicts worse but reports a trustworthy slope wins this job outright.

Trap: the forest cannot do this one

Feature importance is not an effect. It has no units, no direction, and it splits credit between correlated columns close to arbitrarily.

Job 3: screen

The ask. A hospital records 60 things at intake and wants to know which are worth continuing to collect.

- **What matters:** getting from 60 down to a handful, without hand-picking.
- **Reach for:** the lasso. Turn the penalty up and most coefficients go to exactly zero, leaving a shortlist.
- Report it as **a shortlist**, which is what it is.

Trap: the survivors are not effects

They were shrunk on purpose, and they were chosen by looking at the data. If you need defensible numbers, refit an ordinary regression on the survivors and say that you screened first.

Job 4: decide

The ask. A bank has a model that outputs a default probability. Now what?

- **What matters:** a probability you can trust, plus what each kind of mistake costs.
- **Reach for:** anything that outputs a calibrated probability. The model is half the problem.
- The other half is the **threshold**, and that is a business decision, not a statistical one.

Principle: the costs are rarely symmetric

Approving a bad loan and rejecting a good customer do not cost the same amount, so 0.5 is almost never the right cutoff. Unit 11 does this properly.

Job 5: deploy

The ask. This runs every night for two years and someone will call about a specific output.

- **What matters:** stability, few moving parts, and a story for any single prediction.
- **Reach for:** the simplest model that clears the accuracy bar you actually need.
- A model with six tuned hyperparameters is six things that can drift when the data does.

In practice: a strong thing to say

"Boosting was two points better in testing. I shipped the regression, because we have to explain rejections to customers and I would rather be able to."

The five jobs, side by side

Job	Reach for	Judged by	Costs you
Predict	boosting or a forest, regression if rows are scarce	out-of-sample error	coefficients nobody can read
Explain	linear or logistic regression	the interval on one coefficient	accuracy you could have had
Screen	the lasso	does the shortlist hold up	valid inference on the survivors
Decide	anything calibrated, plus a cost table	expected cost, not accuracy	work nobody asked for
Deploy	the simplest thing that clears the bar	does it still work in a year	the last few points of accuracy

Your turn #1: which model, and why

Your turn: two questions for each one

1. A hospital wants to know which of 60 measurements taken at intake are worth continuing to collect.
2. A subscription business wants, for each customer, the chance they cancel next month.
3. A city wants to know whether a new bus lane reduced average commute time.
4. A support team wants to know how many tickets will arrive each hour tomorrow.

For each: **(a)** what would you fit, and **(b)** a random forest would run on all four of these. Would you want it here, and what would it cost you?

Your turn #1: answer

	y is	What I would fit	A forest would run. Why not use it?
1	whatever they track	the lasso, to screen	importances split credit between correlated measurements, and you need a list you can defend
2	yes or no	logistic, *or* a forest	you might well want it. Forest if nobody reads it, logistic if you must explain a decline to a customer
3	a number	regression with the bus lane as the predictor of interest	the ask is the size of one effect with a range on it, and a forest has no such number
4	a count	Poisson regression	a forest predicts counts fine. Poisson gives you a rate you can interpret and extrapolate

Principle: the outcome never decided this

A forest runs on all four. What ruled it in or out was whether anyone needs to read a number out of the model.

The five things you do to any model

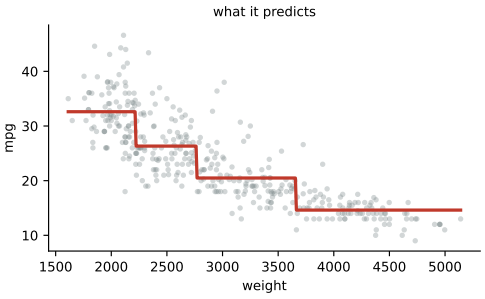
1. **Fit it.** Estimate the parts the data decides.
2. **Tune it.** Set the parts the data does not decide.
3. **Predict with it.** Push a new case through.
4. **Say how wrong it might be.** Attach a range to that prediction.
5. **Judge it against an alternative.** A number alone means nothing.

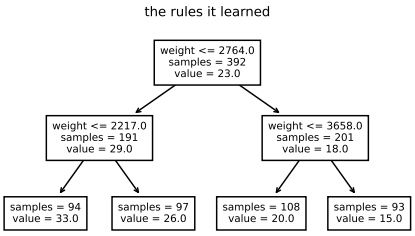
Principle: the list does not change, the tools do

Every family on the menu goes through all five. What differs is which tool does each step, and whether the model can supply it for you or you have to get it from held-out data.

The next five slides walk one regression tree through all of it, on the 392 cars.

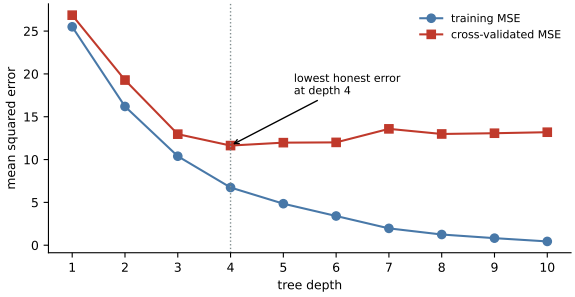
Step 1: fit it





- Fitting a tree means finding the **splits**: which variable, and at what cutoff.
- Here it chose weight at 2764, then 2217 and 3658, giving four groups with average mpg of 33, 26, 20, and 15.
- Those numbers are what the data decided. Nobody chose them.

Step 2: tune it



Depth is not estimated. You choose it, and the data cannot choose it for you by fitting.

- Training error falls forever. By depth 10 it is near zero.
- Honest error bottoms out at **depth 4** and then climbs.

So depth 4, chosen on folds fixed before looking.

Step 3: predict with it

- Fit the depth-4 tree on 294 cars, hold back 98 it never sees.
- Push one held-out car through: it lands in a leaf, and the prediction is that leaf's average.

$$\text{predicted mpg} = 22.8$$

Principle: a tree predicts by lookup

There is no equation to evaluate. The car falls through a sequence of yes/no questions and comes out in a box, and the answer is the average of the training cars already in that box.

Step 4: say how wrong it might be

The tree names no distribution, so it cannot hand you a range. Get one from how badly it missed on the 98 cars it never saw.

- Collect those 98 held-out errors, $y_i - \hat{y}_i$.
- The 5th and 95th percentiles are -4.6 and $+7.6$ mpg.
- Attach them to the prediction.

$$22.8 - 4.6 \;=\; 18.2 \qquad \text{to} \qquad 22.8 + 7.6 \;=\; 30.4 \text{ mpg}$$

That car's actual mpg was **23.0**, comfortably inside. A probability model would have produced this range from its own assumptions instead.

Step 5: judge it against an alternative

Same six predictors, same five folds, three candidates.

Model	Training MSE	Cross-validated MSE
Linear regression	11.59	12.23
Tree, depth 4	6.74	**11.63**
Tree, depth 10	0.44	13.19

Trap: the middle column is a trap

Ranked on training error, the depth-10 tree wins by a mile. On data it did not see, it is the worst of the three, and worse than a straight line.

What "error" means, exactly

For a numeric outcome, the usual measure is **mean squared error**:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2 \qquad \text{RMSE} = \sqrt{\text{MSE}}$$

- Squaring makes big misses count much more than small ones. A miss of 4 counts sixteen times a miss of 1.
- MSE is in squared units, which nobody can picture. RMSE is back in the units of y.
- Our depth-4 tree: MSE $= 11.63$, so RMSE $= 3.41$ mpg. That is the number to quote.

Principle: the measure is a choice

Squared error is a decision that one big miss is worse than several small ones. If that is not true for your problem, use absolute error instead, and say so.

Two ways to get an honest number

Hold out once

Split the rows, fit on most, score on the rest. Simple, and the score depends on which rows happened to be held out.

Cross-validate

Split into k folds. Fit on $k-1$, score on the one left out, rotate, average. Every row gets scored exactly once, by a model that never saw it.

Principle: same rule either way

Anything you learned from the data counts as fitting. Choosing depth, choosing which columns to keep, even choosing the scaling. If it happened before scoring, it has to happen inside the fold.

AIC and BIC: paying for parameters

A **probability model** can report how likely your data was under the fitted model. Call that the likelihood L. Bigger is better, but adding parameters always makes it bigger, so a raw comparison always picks the biggest model.

$$\text{AIC} = 2k - 2\ln L \qquad \text{BIC} = k\ln n - 2\ln L$$

- k is how many parameters you estimated, n is how many rows you have.
- The second term rewards fit. The first charges you for the parameters that bought it.
- **Lower is better.** The raw value means nothing on its own, only differences do.

AIC, worked on the cars

$n = 392$. Three nested models, each fit to the same rows.

Model	k	$\ln L$	**AIC** $= 2k - 2\ln L$
mpg on weight	2	-1130.0	$4 + 2260.0 = 2264.0$
$+$ horsepower	3	-1121.0	$6 + 2242.0 = 2248.0$
$+$ model year	4	-1037.4	$8 + 2074.8 = 2082.8$

- Horsepower buys 16 points of AIC, and model year buys another 165. Both are worth their one parameter.
- BIC charges $\ln(392) = 5.97$ per parameter instead of 2, so it prefers smaller models, and the gap widens as n grows.

Which comparison to use

	AIC and BIC	**Out-of-sample error**
Needs a likelihood	yes, so probability models only	no, works on anything
Data used	all of it, once	held out, or rotated in folds
Cost	one fit per model	k fits per model
Compares across families	no	yes

Trap: comparing numbers that are not comparable

An AIC of 2082 and a cross-validated MSE of 11.63 are different quantities on different scales, and neither one is "better" than the other. Pick one method and use it for every candidate, on identical rows.

Simple or complicated

	Reach for simple	**Reach for flexible**
Goal	explaining, screening, deploying	predicting
Rows	hundreds	tens of thousands and up
Columns	few, chosen for a reason	many, and you do not know which
Audience	someone who will challenge a number	someone who only sees the output

Principle: flexibility is bought with sample size

A complicated model is a bet that you have enough data to pin down everything you just let vary. On 300 rows that bet usually loses.

Three choices, worked

Situation	y **is**	**What I would fit, and why**
1,100 loan applications, 12 clean columns, a regulator reviews every rejection	yes or no	Penalized logistic to pick the columns, then refit an ordinary logistic on the survivors so the coefficients are defensible. Say that you screened first.
6 million ad impressions, 400 columns, only click rate matters	yes or no	Gradient boosting. Same type of y as above and a completely different answer, because nobody will ever read a coefficient.
900 stores, weekly complaint counts, did the new policy reduce them	a count	Poisson regression with a policy indicator. Then ask whether anything else changed that week.

The first two have identical outcome types. The audience decided them, not the data.

The two-model habit

- Fit the simplest defensible model and a flexible one. Both, every time.
- If they agree, report the simple one. Easier to defend, cheaper to maintain, less likely to break when the data shifts.
- If they disagree, you have learned something specific: there is structure the simple model is missing. Go find out what it is.

In practice: a strong thing to say

"The forest beat the regression by four points, so I looked for what it was using, found the threshold at 30 days, added that to the regression, and it caught up. I shipped the regression."

Your turn #2

Your turn: name the family and the reason

1. 60 trial patients, one treatment variable, and the question is whether the drug works.
2. 25,000 sensor readings where you know the relationship bends but not how.
3. A model that has run nightly for a year suddenly gets worse. Which of the five steps do you revisit first?

The reason is the half that gets graded.

Your turn #2: answer

1. **A t-test, or a regression with the treatment indicator.** With $n = 60$ and one variable of interest, a flexible model would spend your data estimating things nobody asked about. Randomization is doing the work here, not the model.
2. **Both.** A flexible fit to see the shape, then name it and put it in a regression you can explain. That is the two-model habit doing its job.
3. **None of them first. Check whether the data still looks like the training data.** Nothing about the model changed, so the thing that changed is outside it. Re-tuning a model against a shifted world just fits the new noise.

LLM check: mistake or not?

LLM check: you asked for help choosing a model

"With 200 patients and 45 predictors I'd recommend a random forest. It handles nonlinearity and interactions automatically and gives you feature importances, so you can report the top predictors as the key risk factors and their p-values."

Three separate problems. Find them before the next slide.

LLM check: answer

- **The ratio.** 200 rows against 45 columns is the corner where flexible models fit well and predict badly. A penalized regression is the honest choice.
- **"Automatically" costs something.** Discovering interactions means spending data to find them. With 200 rows you cannot afford the search.
- **A forest has no p-values.** It names no distribution, so the quantity being asked for does not exist. And an importance score ranks correlated columns close to arbitrarily, so it is not an effect either.

Principle:

The answer is not wrong about what a forest does. It is wrong about whether this dataset can pay for one, and about what the output means.

Choosing a model, in order

1. **Name the job.** Predict, explain, screen, decide, or deploy.
2. **Name the type of y.** A number, a yes or no, a count, a category. Not a judgment call, and it removes most of the menu.
3. **Decide whether you need parameters that mean something.** If yes, you are in the probability column and that is the end of the discussion.
4. **Count what you have against what you are estimating.** Rows, columns, and noise.
5. **Fit the simple one first.** It is your baseline and often your answer.
6. **Fit a flexible one on the same folds**, compare with one method, and take the simpler model when they are close.
7. **Report on data you touched once**, and say what else you tried.

The traps, all in one place

Trap: the ones that bite most often

- Choosing the algorithm before naming the job.
- Forcing a linear model onto a count or a yes/no outcome.
- Judging a model on the rows it was fitted to.
- Asking an algorithmic model for a p-value or an AIC.
- Reading a penalized or shrunk coefficient as an effect.
- Comparing an AIC against a cross-validated error.
- Tuning a hyperparameter until the answer looks right.

What you should be able to do with software

Nobody is asking you to memorize syntax. You are expected to know which procedure the question calls for, what to hand it, and what the output means.

You should be able to	**What you would reach for**
Fit a linear, logistic or Poisson model	`smf.ols`, `smf.logit`, `smf.glm`
Read the coefficient table	`fit.summary()`
Fit a tree, a forest, or boosting	`DecisionTree...`, `RandomForest...`
Tune a hyperparameter honestly	`GridSearchCV` over folds fixed first
Compare probability models	`fit.aic`, on identical rows
Compare anything at all	`cross_val_score` with one shared `KFold`
Screen many columns	`LassoCV`, then read what survived

Where we go next

- You now have the map. The rest of Part I walks the parts of it that matter most.
- **Unit 3** takes the linear model seriously: reading a slope, what happens when you add a variable, and how to tell whether the fit is real.
- **Unit 4** comes back to tuning and honest evaluation properly, and **Unit 5** to putting a range on a prediction.

Principle: the through-line

The model is a means. The job you were given decides which one, and saying that job out loud is most of the work.